

21. Explainable AI Module

21.1 Why Explanation Is Required Here

A risk band on its own is not actionable. Told that they are *High* risk, a user learns nothing they can act upon; told that the assessment rests chiefly on a savings rate of 4% and an emergency fund covering under three months, they learn what to change.

There is a second reason, particular to the domain. The rule-based rubrics that this system replaces, described in Section 8.5, are transparent by construction: an analyst can read the weight attached to each answer. Substituting a learned function forfeits that transparency unless something restores it. The explainability module exists to pay that debt, not to decorate the output.

21.2 Method

Attributions are computed with SHAP (SHapley Additive exPlanations), which assigns each feature the average marginal contribution it makes to the prediction across all orderings in which features could be introduced. The construction is the Shapley value from cooperative game theory, and it is the unique attribution satisfying local accuracy, missingness and consistency.

Computing it exactly is exponential in the number of features in general. For tree ensembles it is not: `TreeExplainer` exploits the tree structure to compute exact Shapley values in time polynomial in the depth and number of trees. Since the deployed model is a 200-tree Random Forest of depth 10 over 48 features, exact attribution is tractable per request, and no sampling approximation is needed.

`KernelExplainer` is retained in the code for the neural network, which is not deployed.

21.3 Global and Local Explanation

Two distinct artefacts are produced, and conflating them is a common error.

Global importance is computed once, during training, by averaging the absolute SHAP value of each feature across the test partition. It answers: on which features does this model rely, in aggregate? The result is stored in `model_metadata.json` and rendered as the beeswarm plot at `ml/plots/shap_summary.png`.

Local attribution is computed per request, for the single submitted row. It answers: for *this* person, which of their inputs drove *their* assessment? This is what the results page displays, and it is the reason SHAP is invoked at prediction time rather than merely at training time.

A user with an unusual profile can receive a local explanation that departs sharply from the global ranking, and that is the point.

21.4 The Serving Path Discards Direction

The implementation does not deliver the local explanation it promises.

`TreeExplainer.shap_values` on a multiclass model returns an array of shape `(n_samples, n_features, n_classes)` : a signed contribution from every feature toward every class. The serving code reduces it as follows:

```
sv_row = np.abs(sv[0]).mean(axis=1) # (n_features,)
```

Two operations occur here, and each destroys information.

The absolute value is taken, so a feature that pushed the prediction *away* from the assigned class becomes indistinguishable from one that pushed it *toward* that class. The mean is then taken across the three classes, so an attribution that was specific to *High* is averaged with those for *Low* and *Medium*, yielding a class-agnostic magnitude.

What the API returns is therefore not "how did this feature affect my risk assessment" but "how much did this feature matter to the model at this point, in any direction, for any class". Empirically, across 120 attributions drawn from twelve varied inputs, **not one is negative**.

The consequence propagates to the interface. `Results.tsx` colours each bar blue for a positive attribution and red for a negative one, and captions the chart "*Blue = increases risk score, Red = decreases risk score.*" No bar can ever be red. The legend describes a distinction that the data it renders does not carry, and a user reading it would reasonably conclude that every one of their top ten features increased their risk — including, in the sample below, a high credit score.

```
financial_discipline_score +0.0455
savings_rate               +0.0162
credit_score               +0.0159
emergency_fund_months      +0.0119
savings_ratio              +0.0108
```

For this record — savings rate 30%, credit score 720, three months of emergency fund — the model predicted *Medium*. A signed, class-specific attribution would show `credit_score` and `savings_rate` pulling *away* from *High*. The magnitude presentation cannot express that, and the colour legend actively misleads.

The repair is small. Selecting the column corresponding to the predicted class and preserving its sign, rather than averaging absolute values across classes, yields the attribution the interface is already written to display:

```
k = int(model.predict(X_df)[0]) # index of predicted class
sv_row = sv[0][:, k]           # signed, class-specific
```

This was identified while writing this chapter and is not applied, because doing so would change every SHAP figure reported in Sections 14 and 22 and require regenerating them. It is recorded as a defect in Section 26, and it is the first thing that should be fixed in any continuation of this work.

The global ranking in `shap_summary.png`, computed by `train.py` over the test partition, is unaffected: averaging absolute values across a population is the correct way to summarise global importance, and that is what a beeswarm plot displays.

21.5 What the Explanations Show

Subject to Section 22.5 — that three of the top five features encode a single underlying quantity — the ranking is consistent with the generating function.

Rank	Feature	Mean abs. SHAP	Appears in <code>risk_score</code> ?
1	savings_rate	0.0462	yes, weight -30
2	financial_discipline_score	0.0407	derived from three of the five
3	emergency_fund_months	0.0280	yes, weight +10 (as an indicator)
4	savings_ratio	0.0217	proxy for savings_rate
5	expense_ratio	0.0217	complement of savings_ratio
6	investment_experience_years	0.0147	yes, weight -15
7	behavioral_composite_score	0.0134	partly
8	credit_score	0.0130	yes, weight +25
9	years_of_experience	0.0039	no
10	age	0.0033	no

Every feature that enters `risk_score` appears in the top eight, and the two that do not — `years_of_experience` and `age` — carry an order of magnitude less attribution. The model has recovered the structure it was given.

`credit_score` at rank eight is lower than its weight of +25 would suggest. The explanation lies in preprocessing: the IQR clip described in Section 9.3 truncates `credit_score` at a lower bound of 660.5, discarding the variation among poor-credit individuals precisely where that feature is most discriminative. The model cannot use information that was removed before it saw the column.

No lifestyle feature reaches the top ten. This is not a finding about human behaviour. `risk_score` is a function of five financial quantities only, so the lifestyle attributes have nothing to contribute, and Section 13.6 anticipates this. A real dataset might behave very differently, and nothing here licenses a claim either way.

21.6 Behavioural Biases and Recommendations

The interface also presents a list of detected behavioural biases and a set of written recommendations. Neither is produced by the model.

Both are generated by fixed rules over the raw inputs and the predicted class — for instance, flagging *present bias* when the savings rate falls below 10% while the subscription count exceeds five, or *loss aversion* when a Low-risk assessment coincides with substantial investment experience. Nine bias rules and eleven recommendation branches are implemented, of which three to five typically fire for a given record.

These are heuristics drawn from the behavioural-finance vocabulary of Section 3, not inferences. They are deterministic given the input, they were not validated against any labelled data, and no claim is made that a user so flagged exhibits the bias named. They are presented in the report as an illustrative layer, and they

are the component of this system with the weakest evidentiary basis.

Figure 21.1 — *Global SHAP beeswarm*. Already generated at `ml/plots/shap_summary.png` . Place in Section 21.5.

Figure 21.2 — *Top-20 features by mean absolute SHAP*. Already generated at `ml/plots/feature_importance.png` . Place alongside Figure 21.1.

Screenshot 21.1 — *Local SHAP bar chart on the results page*, with the "Blue = increases risk" legend visible. Caption it to state that no bar is red, and cross-reference Section 21.4. Used honestly, this screenshot documents the defect rather than the feature.

Figure 21.3 — *Intended versus actual attribution for one record*. Two side-by-side bar charts: the signed, class-specific SHAP values that `TreeExplainer` produces, and the absolute cross-class means that the API returns. Generate by calling `TreeExplainer` directly on the sample in Section 21.4. This figure makes the defect legible in one glance. Place in Section 21.4.